

A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval- Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE FIVE

Basic Retrieval Techniques

Beginner to Industry Level • Assumes only basic Python

MODULE FIVE

Basic Retrieval Techniques

The retriever is the single component that most determines whether a RAG system gives good answers — because the generator can only ever reason over the chunks the retriever hands it. This module opens the query phase and turns your vector store into a genuinely effective retriever. You will master plain similarity search and its parameters, learn to use score thresholds so the system can honestly say "I don't know," understand Maximal Marginal Relevance for balancing relevance against diversity, and grasp the crucial distinction between dense and sparse retrieval — culminating in hybrid search and ensemble retrievers that combine the best of both.

WHERE WE ARE

Modules 2–4 completed the indexing phase: you can load, split, embed, and store documents in a fast, persistent vector database. This module begins the **query phase**. It takes the vector store you built and wraps it in a *retriever* — the component that, for any user question, returns the chunks worth reasoning over. Module 6 then makes that retriever smarter; this module makes it solid.

5.1 Introduction

Every module so far has been preparation. You can now turn a folder of messy documents into a clean, embedded, persistent, searchable knowledge base. But a knowledge base is not an answer-giving system. It is a library with the lights on and no librarian. This module hires the librarian.

The **retriever** is the component that stands between a user's question and the knowledge base. Its job is deceptively simple to state: given a question, return the chunks most worth showing the language model. Yet this one component carries more responsibility for answer quality than any other, for a reason worth repeating until it is instinct: **the generator can only reason over what the retriever gives it**. A brilliant language model handed three irrelevant chunks will produce a confident, fluent, wrong answer. The same model handed the one correct chunk will produce a correct one. Retrieval is the bottleneck.

In Module 1 you used the most basic retriever — "*return the top 4 most similar chunks*" — and it worked well enough to see RAG function. That basic retriever, though, has real limitations that you will hit the moment you build something for actual users:

- It returns k chunks **no matter what** — even when the knowledge base contains nothing relevant at all, it still confidently hands back its four least-bad chunks.
- It can return four chunks that are **near-duplicates of each other**, wasting the context window on redundancy and starving the model of breadth.
- It searches purely by **meaning**, which means it can stumble on exact terms — a product code, an error number, a person's name, a piece of jargon — that a user typed precisely and expects to be matched precisely.

This module fixes each of these. We will cover similarity search and its parameters properly, introduce score thresholds so a retriever can admit ignorance, explain Maximal Marginal Relevance to eliminate redundancy, draw the fundamental line between dense (semantic) and sparse (keyword) retrieval, and combine them into hybrid search and ensemble retrievers — the most reliable basic-retrieval setup in production today.

5.2 Why This Topic Matters

WHY THIS MATTERS

If a RAG system gives a wrong or unhelpful answer, the cause lies in retrieval far more often than in the language model. The model is rarely "dumb"; it is usually *under-informed* — it never received the chunk it needed, or it received noise. This module is therefore where you gain the most direct leverage over answer quality. It is also where a great deal of practical RAG knowledge lives: "how do you handle a query with no good answer?", "what is MMR?", and "what is hybrid search?" are all standard interview questions, and all are answered here.

Four concrete reasons this module repays close study:

1. **Retrieval quality sets the ceiling on the whole system.** No prompt and no model can recover an answer from evidence the retriever never fetched.
2. **The basic retriever has honest limitations.** Knowing them — always-returns- k , redundancy, the keyword blind spot — is what lets you choose the right fix instead of guessing.
3. **Hybrid search is a production default.** Combining semantic and keyword retrieval is one of the highest-return, lowest-risk improvements you can make, and you should understand exactly why it works.
4. **It is the foundation for everything advanced.** Module 6's smart retrievers and Module 7's advanced patterns are all built on the retriever interface and the dense/sparse ideas established here.

5.3 A Beginner-Friendly Explanation

A **retriever**, in LangChain, is simply an object with one job: you give it a text query, and it gives you back a list of relevant `Document` chunks. That is the entire contract. Everything in this module is a different *strategy* for fulfilling that contract well.

The most natural strategy is the one you already met: **similarity search**. Embed the query, find the chunks whose vectors are nearest, return them. It is the default, and it is good. But "good" is not "good enough for real users," and it helps to see *why* by naming three everyday failures in plain language.

Failure one — the retriever cannot say "I don't know." Ask a basic retriever a question about a topic that simply is not in the knowledge base, and it does not shrug. It still returns k chunks — the k closest it has — even if "closest" means "still completely unrelated." It then hands those unrelated chunks to the model, which dutifully tries to answer from them. The user gets a wrong answer instead of an honest "that is not in my documents." The fix is a **score threshold**: a minimum similarity below which a chunk is treated as "not actually a match."

Failure two — the retriever returns the same thing four times. Suppose your knowledge base contains five chunks that all say roughly "the refund window is 30 days," phrased slightly differently. Ask about refunds, and similarity search happily returns the four most similar chunks — which are those four near-identical refund chunks. The model now has four copies of one fact and zero information about anything else related to the question. The fix is **Maximal Marginal Relevance**, which deliberately picks chunks that are relevant *and* different from each other.

Failure three — the retriever misses exact words. Semantic search is brilliant at meaning and, paradoxically, can be weak at *exactness*. If a user asks about error code `ERR_4012` or product `Model X-450` or an employee named `Priya Nair`, the embedding of that query captures its general *meaning* but may not strongly prioritise the precise token. A chunk that mentions `ERR_4012` exactly might be ranked below a chunk that is "semantically about errors" in general. The fix is to *also* run an old-fashioned **keyword search** and combine the two — **hybrid search**.

So this module is, in plain terms: take the basic retriever, then teach it to admit ignorance (thresholds), avoid repeating itself (MMR), and respect exact words (hybrid search).

DEFINITION

A **retriever** is a component that takes a text query and returns a list of relevant document chunks. **Dense retrieval** finds chunks by semantic similarity of embeddings (search by *meaning*). **Sparse retrieval** finds chunks by overlap of exact words (search by *keyword*). **Hybrid search** combines both.

5.4 Real-World Analogy: Three Librarians

REAL-WORLD ANALOGY

Imagine three librarians, and a reader who asks: *"I need material on recovering a locked account — something about error E-77."*

The keyword librarian (sparse retrieval) scans the catalogue for the literal words. They will instantly find every book that mentions "error E-77" — that exact code is their strength. But ask for "recovering a locked account" and, if the relevant book is titled "regaining access to a suspended profile," they miss it entirely, because the words do not match.

The meaning librarian (dense retrieval) understands intent. They grasp that "recovering a locked account" and "regaining access to a suspended profile" are the same idea, and find that book easily. But the precise code "E-77" is, to them, just a vague technical detail; they may hand you a great general book on account recovery that never mentions E-77 at all.

The expert librarian (hybrid search) does both at once: they understand your intent *and* they note the exact code. They consult both lists, weigh them together, and hand you the book that is both about the right concept and mentions E-77 precisely. This is why hybrid search outperforms either approach alone.

The analogy extends to the other two techniques. A good librarian also (a) tells you honestly *"we have nothing on that"* rather than handing you an irrelevant book just to hand you something — that is the **score threshold** — and (b) when several books cover the same point, gives you *different* useful books rather than four near-copies of the same one — that is **MMR**.

5.5 Core Concepts: The Retriever in the Query Phase

Recall the query phase from Module 1: embed the query, search, augment the prompt, generate, return. This module owns the **search** step, and in LangChain that step is performed by a **retriever**.

The retriever is an *abstraction*. Your code asks a retriever for relevant chunks and does not care *how* it found them — by dense search, sparse search, or a blend. This is why LangChain exposes every retrieval strategy through the same interface:

```
chunks = retriever.invoke("How many paid leave days do I get?")
# → returns a list[Document], regardless of the strategy inside
```

Because the interface is uniform, you can swap a basic similarity retriever for an MMR retriever or a hybrid ensemble retriever **without changing a single line of the chain** that consumes it. The retriever is a pluggable component — and this module is a tour of what you can plug in.

A retriever is almost always created *from* a vector store, by calling `.as_retriever()` and passing configuration. The two ideas — vector store and retriever — are related but distinct: the **vector store** is the storage-and-search engine; the **retriever** is the strategy layer that decides *how* to use it and *what* to return.

5.6 Similarity Search Fundamentals

The default retriever uses plain **similarity search**: embed the query, find the k nearest chunk vectors by the distance metric (cosine, from Module 3), return them.

```
retriever = vector_store.as_retriever(
    search_type="similarity",      # the default
    search_kwargs={"k": 4},       # return the 4 nearest chunks
)
chunks = retriever.invoke("What is the refund policy?")
```

The single most important parameter is **k** — how many chunks to return. It is a real trade-off:

- **k too low** (e.g. 1): if the one chunk returned is not the right one, or if the answer is spread across several chunks, the model has nothing to work with. Low recall.
- **k too high** (e.g. 20): the prompt fills with chunks of decreasing relevance. The genuine answer is buried among noise that can distract the model, and you pay for far more tokens. The "lost in the middle" effect — models attending less to content in the middle of a long context — also begins to bite.
- **The sweet spot** for most systems is **k between 3 and 8**. Start at 4 and tune by measurement (Module 9).

COMMON MISTAKE

Believing that a larger **k** is always safer because "more context can't hurt." It can. Padding the prompt with marginally relevant chunks dilutes the signal, invites the model to latch onto an off-topic passage, raises token cost, and worsens the "lost in the middle" problem. Retrieval quality is about precision, not volume. The goal is the *few* chunks that genuinely matter, not the *most* chunks you can fit.

5.7 Similarity Score Thresholds

Plain similarity search has a structural flaw: it **always returns k chunks**, regardless of whether any of them are actually relevant. If a user asks about a topic absent from the knowledge base, the retriever does not return an empty list — it returns its k least-distant chunks, which may all be genuinely unrelated. The model then receives irrelevant context and, lacking any instruction to the contrary, often answers anyway. The result is a

confident hallucination caused not by the model but by the *retriever's refusal to admit it found nothing*.

The fix is a **similarity score threshold**. Every retrieved chunk comes with a similarity score (Module 4's `similarity_search_with_score`). A threshold says: *discard any chunk whose score is below this minimum*. If every candidate falls below the threshold, the retriever correctly returns **nothing**, and the system — with a properly grounded prompt — can honestly reply *"I could not find that in the documents."*

```
retriever = vector_store.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={
        "score_threshold": 0.5,  # discard chunks scoring below 0.5
        "k": 4,
    },
)
```

The hard part is **choosing the threshold value**, and it cannot be guessed in the abstract for two reasons. First, score *semantics* differ — some stores report a similarity (higher is better, often 0-1), others a distance (lower is better) — so you must know which your store reports and which direction "good" is. Second, the *right* number depends on your data and embedding model. The correct procedure is empirical: run a batch of queries you *know* have good answers and a batch you *know* do not, look at the score distributions of each, and pick a threshold that cleanly separates them.

BEST PRACTICE

Set the threshold by measurement, not intuition. Collect scores from "should-match" queries and "should-not-match" queries, plot or tabulate both distributions, and choose a cut-off in the gap between them. Set it too high and you reject genuine answers (low recall — the system says "I don't know" when it actually does); set it too low and irrelevant chunks leak through (low precision — hallucination returns). The threshold is the dial that lets a RAG system *fail honestly* instead of failing confidently — one of the most valuable properties a production system can have.

5.8 Maximal Marginal Relevance (MMR)

5.8.1 The Redundancy Problem

Plain similarity search ranks chunks by one criterion only: similarity to the query. It has no awareness of how the chosen chunks relate *to each other*. So when several chunks all express the same relevant idea, similarity search returns *all of them* — they are, after all, all highly similar to the query. You ask for the top 4 and receive four near-duplicates of a single fact.

This wastes the most precious resource in RAG: the slots in the context window. Four redundant chunks give the model one fact stated four times and crowd out four *different* chunks that could have covered other facets of the question. Redundancy is silently expensive.

5.8.2 How MMR Works

Maximal Marginal Relevance (MMR) is a retrieval strategy that selects chunks by balancing **two** criteria instead of one: a chunk should be *relevant to the query* **and** *different from the chunks already selected*.

MMR builds its result set one chunk at a time. At each step it asks: among the remaining candidates, which one offers the best combination of *high relevance to the query* and *low similarity to what I have already picked*? It then adds that chunk and repeats. The effect is a result set that is both on-topic and varied — it covers the *breadth* of the question rather than hammering one point.

A single parameter, often called `lambda_mult`, controls the balance. Conceptually:

```
MMR score of a candidate chunk =
    λ · (similarity of chunk to the QUERY)
    - (1 - λ) · (greatest similarity of chunk to ANY
                  already-selected chunk)

λ = 1.0 → pure relevance    (behaves like plain similarity search)
λ = 0.0 → pure diversity    (ignores the query, maximises difference)
λ ≈ 0.5 → balanced          (relevant AND varied – the usual choice)
```

In LangChain, MMR is a search type with two key parameters: `fetch_k`, how many candidate chunks to pull in initially, and `lambda_mult`, the balance dial. MMR fetches `fetch_k` candidates by similarity, then *re-selects* the final `k` from among them using the relevance-versus-diversity score.

```
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={
        "k": 4,                # final number of chunks to return
        "fetch_k": 20,         # candidates to consider before re-selecting
        "lambda_mult": 0.5,    # 0 = max diversity, 1 = max relevance
    },
)
```

WHY THIS MATTERS

MMR matters most for *broad* or *multi-faceted* questions. Ask "what are the company's leave policies?" — a question with several parts (annual leave, sick leave, parental leave) — and plain similarity search may return four chunks all about annual leave because that section is the densest match. MMR instead returns one strong chunk per facet, giving the model the full picture. For a narrow, single-fact question, MMR and plain similarity behave almost identically. So reach for MMR when query breadth, or a knowledge base with repetitive content, makes redundancy likely.

5.9 Dense vs Sparse Retrieval

We now reach the most conceptually important distinction in this module. There are two fundamentally different ways to find relevant text, and understanding both is essential.

5.9.1 Dense Retrieval (Semantic Search)

Dense retrieval is everything you have done so far. It uses the dense embeddings of Module 3: the query and the chunks are converted to dense vectors, and similarity is measured in that meaning-space.

- **Its strength is meaning.** It matches concepts regardless of vocabulary — "how do I get my money back" finds "Refund Policy." It handles synonyms, paraphrase, and intent.
- **Its weakness is exactness.** A precise, rare token — a product code, an error number, a proper name, a piece of niche jargon — is, to an embedding, just one small contributor to an overall meaning. Dense retrieval can rank a chunk that is "generally about the topic" above the chunk that contains the *exact term the user typed*.

5.9.2 Sparse Retrieval (Keyword Search) and BM25

Sparse retrieval is the older, lexical approach: find chunks that contain the *same words* as the query. It is called "sparse" because, as in Module 3's one-hot discussion, it represents text as a very high-dimensional, mostly-zero vector over the whole vocabulary — one slot per word, non-zero only for the words actually present.

The standard, battle-tested sparse retrieval algorithm is **BM25**. You do not need its formula, but you do need its three intuitions, because they explain exactly when it wins:

1. **Term frequency** — a chunk that uses a query word *more often* is scored higher for that word.
 2. **Inverse document frequency** — a word that is *rare across the whole corpus* counts for more than a common word. Matching "the" is worthless; matching "E-77" is highly informative. This is why BM25 excels at distinctive tokens.
 3. **Document length normalisation** — long chunks are prevented from scoring highly just by virtue of containing more words.
- **BM25's strength is exactness.** Codes, IDs, names, acronyms, exact phrases, rare technical terms — BM25 matches them precisely and reliably, because it is literally looking for those words.
 - **BM25's weakness is meaning.** It has no concept of synonyms or paraphrase. "Car" and "automobile" are, to BM25, completely unrelated words. If the query and the answer use different vocabulary for the same idea, BM25 misses it entirely.

DEFINITION

BM25 (Best Matching 25) is a classic keyword-ranking algorithm that scores a chunk against a query using term frequency, inverse document frequency (rarer words matter more), and document-length normalisation. It is a **sparse** retrieval method: it matches exact words, not meaning.

5.9.3 Dense and Sparse Are Complementary

The crucial insight: **dense and sparse retrieval fail in opposite situations**. Dense retrieval misses exact terms but understands meaning; sparse retrieval misses meaning but nails exact terms. Their weaknesses do not overlap — which is precisely what makes combining them so powerful.

Aspect	Dense retrieval (semantic)	Sparse retrieval (BM25)
Matches on	Meaning / concepts	Exact words
Handles synonyms & paraphrase	Yes	No
Handles exact codes, IDs, names	Weakly	Excellently
Needs an embedding model	Yes	No
Out-of-vocabulary / rare terms	Struggles	Handles well
Typical failure	Misses a precise token	Misses a paraphrased concept

5.10 Hybrid Search

Hybrid search runs *both* a dense retriever and a sparse retriever for the same query and **combines** their results into a single ranked list. Because the two methods fail in opposite situations, their combination is more robust than either alone — it understands meaning *and* respects exact words.

The one subtlety is *how* to combine two result lists, because their scores are not comparable: a cosine similarity (roughly 0–1) and a BM25 score (an unbounded positive number) cannot simply be added. The standard solution is **Reciprocal Rank Fusion (RRF)**, which ignores the raw scores and combines results by their *rank position* in each list — a chunk that ranks highly in *either* list, or moderately in *both*, rises to the top of the merged list. RRF is robust precisely because rank is comparable across methods even when scores are not.

LOOKING AHEAD

Reciprocal Rank Fusion is introduced here only as much as you need to understand hybrid search. It is treated in full — including the formula and its role in *RAG Fusion* — in Module 7. For now, hold this one idea: **RRF merges multiple ranked lists by rank position rather than by raw score.**

In LangChain, hybrid search is most commonly assembled with an **ensemble retriever** (next section). Some vector databases — Weaviate, Qdrant, Pinecone, Milvus — also offer *native* hybrid search built into the database itself; when available, it can be more efficient. Either way, the concept is identical: combine semantic and keyword retrieval.

5.11 Ensemble Retrievers

An **ensemble retriever** is LangChain's general tool for combining several retrievers into one. You give it a list of retrievers and a list of **weights**; it runs all of them, fuses their results with RRF, and returns one merged list — all behind the standard retriever interface.

The most common ensemble — and the standard way to build hybrid search in LangChain — is a **BM25 retriever + a dense vector retriever**:

```
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

# Sparse retriever (keyword / BM25) – built directly from the chunks
bm25_retriever = BM25Retriever.from_documents(chunks)
bm25_retriever.k = 4

# Dense retriever (semantic) – built from the vector store
dense_retriever = vector_store.as_retriever(search_kwargs={"k": 4})

# Combine them into one hybrid retriever
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, dense_retriever],
    weights=[0.5, 0.5],          # equal trust in keyword and semantic
)
chunks = hybrid_retriever.invoke("How do I fix error E-77 when logging in?")
```

The **weights** let you tune how much each retriever's ranking influences the fused result. `[0.5, 0.5]` trusts both equally — a sensible default. If your domain is heavy with exact codes and identifiers, weight BM25 higher; if it is conceptual prose where users paraphrase freely, weight the dense retriever higher. As always, tune by measurement.

INTERVIEW INSIGHT

"What is hybrid search and why is it better than pure semantic search?" is a very common interview question. The complete answer: hybrid search combines **dense** retrieval (semantic, matches meaning, handles synonyms and paraphrase) with **sparse** retrieval such as **BM25** (keyword, matches exact words, handles codes, IDs, and names). It is better because the two methods *fail in opposite situations* — dense misses precise tokens, sparse misses paraphrased concepts — so combining them covers both. The results are merged with **Reciprocal Rank Fusion**, which fuses by rank position because the two methods' raw scores are not comparable. In LangChain it is built with an **EnsembleRetriever** over a **BM25Retriever** and a vector-store retriever.

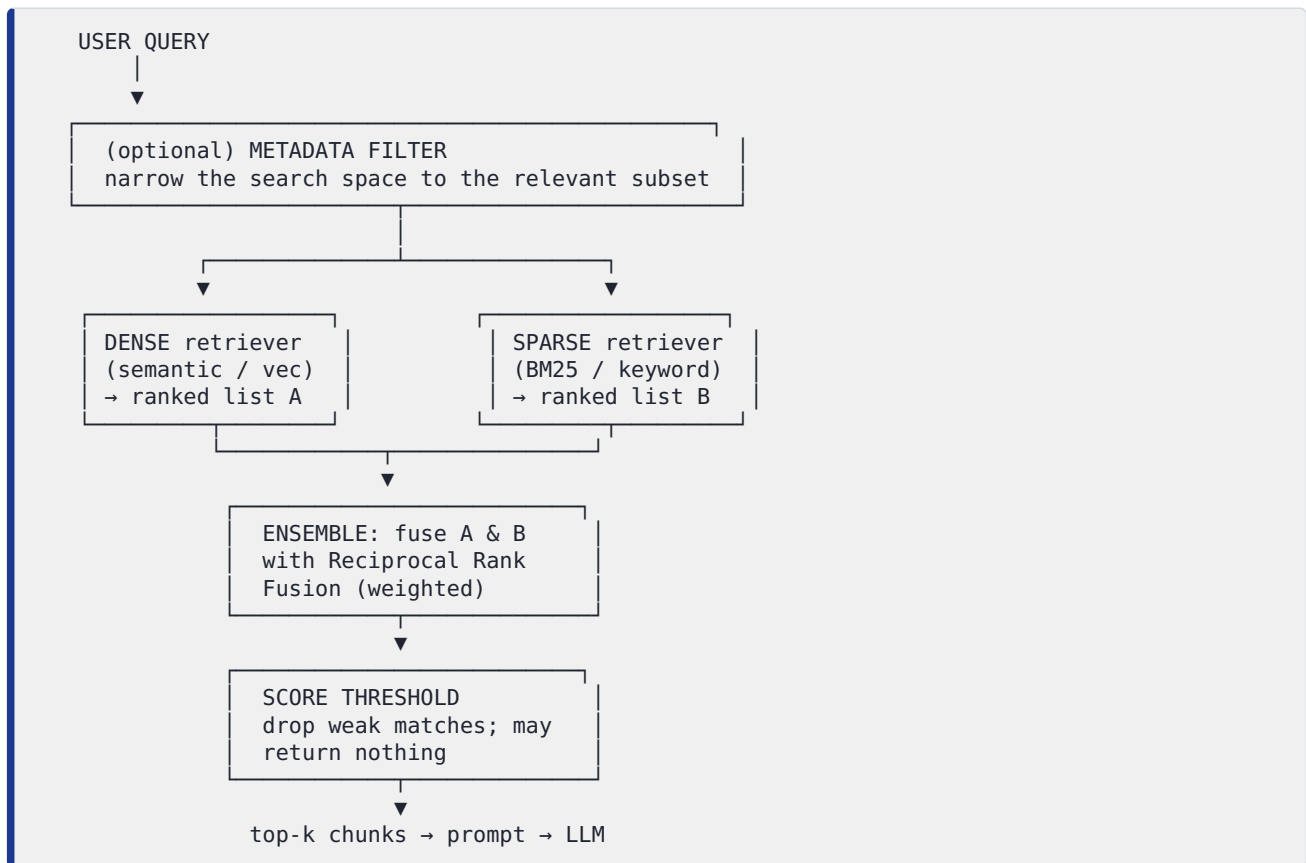
5.12 Retrieval Optimisation Strategies

Beyond choosing a strategy, several practical levers improve any retriever:

- **Tune `k` deliberately.** Start at 3–8; measure; do not pad the prompt with marginal chunks.
- **Use thresholds to fail honestly.** A retriever that can return nothing prevents a class of hallucination.
- **Use MMR for broad or repetitive content.** Diversity in the result set covers multi-faceted questions and avoids near-duplicate chunks.
- **Default to hybrid search** unless you have a clear reason not to — it is a low-risk, high-return upgrade over pure semantic search.
- **Filter by metadata** (Module 2) to shrink the search space before retrieval — it raises precision and speed at once.
- **Get chunking right upstream.** Retrieval quality is bounded by chunk quality (Module 2); a retrieval problem is sometimes a chunking problem in disguise.
- **Re-rank the candidates.** Fetch a larger candidate set, then re-order it with a more powerful model — *contextual compression* and re-ranking are covered in Module 6.
- **Improve the query itself.** Rewriting or expanding the user's query before retrieval — *query transformation* — is covered in Module 6.
- **Measure everything.** "Better" retrieval is meaningless without metrics; Module 9 (RAGAS) makes it rigorous.

5.13 Workflow: A Practical Retrieval Pipeline

A solid basic-retrieval setup, assembled from this module, looks like this:



This pipeline understands meaning and exact words (hybrid), can refuse to answer when nothing fits (threshold), and — by swapping the dense retriever's search type to MMR — can also avoid redundancy. It is a genuinely strong baseline, and Module 6 will sharpen each stage further.

5.14 Step-by-Step Implementation and Code

We will build all four strategies and compare them on the same knowledge base. Save the following as `module5_retrieval.py`.

```

"""
module5_retrieval.py
Similarity search, score thresholds, MMR, and hybrid (ensemble) retrieval.
Stack: Python + LangChain + Gemini + FAISS + BM25.
Install: pip install langchain langchain-community langchain-google-genai \
        faiss-cpu rank-bm25 python-dotenv
"""

import os
from dotenv import load_dotenv
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever
from langchain_core.documents import Document

load_dotenv()
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

# A small knowledge base with deliberate redundancy and an exact code
docs = [
    Document(page_content="Full-time staff get 24 days of paid annual leave."),
    Document(page_content="Paid yearly vacation for permanent employees is 24 days."),
    Document(page_content="Annual leave entitlement for full-time staff: 24 days."),
    Document(page_content="Sick leave is granted for up to 10 days per year."),
    Document(page_content="Parental leave of 16 weeks is available to new parents."),
    Document(page_content="To resolve login error E-77, clear the browser cache."),
]
vector_store = FAISS.from_documents(docs, embeddings)

# ----- 1. PLAIN SIMILARITY SEARCH -----
similarity_retriever = vector_store.as_retriever(
    search_type="similarity", search_kwargs={"k": 3}
)

# ----- 2. SIMILARITY WITH A SCORE THRESHOLD -----
threshold_retriever = vector_store.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.5, "k": 3},
)

# ----- 3. MAXIMAL MARGINAL RELEVANCE (MMR) -----
mmr_retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 3, "fetch_k": 6, "lambda_mult": 0.5},
)

# ----- 4. HYBRID SEARCH (dense + sparse ensemble) -----
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 3
dense_retriever = vector_store.as_retriever(search_kwargs={"k": 3})
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, dense_retriever],
    weights=[0.5, 0.5],
)

def show(label, retriever, query):
    print(f"\n=== {label} === query: '{query}'")
    results = retriever.invoke(query)
    if not results:
        print(" (no chunks passed the threshold – system should say"
              " 'I don't know')")
    for i, doc in enumerate(results, 1):
        print(f" {i}. {doc.page_content}")

if __name__ == "__main__":

```

```
# Redundancy test: a broad query over a base with near-duplicate chunks
show("Plain similarity", similarity_retriever, "annual leave")
show("MMR (diverse)", mmr_retriever, "annual leave")

# Exact-code test: dense search may miss 'E-77'; hybrid should not
show("Plain similarity", similarity_retriever, "fix error E-77")
show("Hybrid (dense+BM25)", hybrid_retriever, "fix error E-77")

# Out-of-knowledge test: nothing relevant exists
show("Threshold retriever", threshold_retriever,
     "what is the company stock price")
```

5.15 Line-by-Line Code Explanation

Setup and knowledge base. As before, the Gemini embedding model is created once. The six `Document` objects are crafted deliberately to expose each technique: the *first three* chunks all state the same 24-days-leave fact in different words (to demonstrate redundancy and MMR), and the *last* chunk contains an exact code, `E-77` (to demonstrate the keyword blind spot and hybrid search). `FAISS.from_documents` builds the dense store.

Retriever 1 — plain similarity. `search_type="similarity"` with `k=3` is the basic retriever. With this knowledge base, asking about "annual leave" will return the *three near-identical leave chunks* — a live demonstration of the redundancy problem.

Retriever 2 — score threshold. `search_type="similarity_score_threshold"` adds `score_threshold=0.5`: any chunk scoring below 0.5 is discarded. For a query about something genuinely absent — the company stock price — every candidate should fall below the threshold and the retriever returns an **empty list**, which is the correct, honest behaviour.

Retriever 3 — MMR. `search_type="mmr"` with `fetch_k=6` and `lambda_mult=0.5`. MMR first pulls 6 candidates by similarity, then re-selects the final 3 to balance relevance against diversity. Asked about "annual leave," it will *not* return three copies of the leave fact; it returns one leave chunk plus other distinct, relevant chunks (sick leave, parental leave) — covering the breadth of the topic.

Retriever 4 — hybrid ensemble. `BM25Retriever.from_documents(docs)` builds a sparse keyword retriever directly from the chunks — note it needs the chunks themselves, not the vector store, and requires no embedding model. `dense_retriever` is the familiar semantic retriever. `EnsembleRetriever` combines them with equal `weights=[0.5, 0.5]`, fusing their results with Reciprocal Rank Fusion. For the query "fix error E-77," BM25 will rank the `E-77` chunk highly because it matches that exact, rare token, ensuring it surfaces even if dense search alone would have under-ranked it.

The `show` helper and the tests. `show` runs a retriever and prints its chunks, explicitly flagging the empty-result case. The `_main_` block runs three targeted comparisons: a **redundancy test** (plain similarity vs MMR on a broad query — watch MMR diversify), an

exact-code test (plain similarity vs hybrid on `E-77` — watch hybrid catch the precise token), and an **out-of-knowledge test** (the threshold retriever on an absent topic — watch it correctly return nothing). Running this file makes all four techniques visible and concrete in a single execution.

BEST PRACTICE

The `BM25Retriever` is built from the same chunk list that produced the vector store, so the two retrievers always search the *same* knowledge. In a real system, keep this in sync: whenever you re-ingest documents, rebuild both the vector store and the BM25 index together. A hybrid retriever whose sparse and dense halves disagree about what is in the knowledge base produces confusing, hard-to-debug results.

5.16 Common Mistakes

- **Assuming the model is at fault for a bad answer.** Retrieval is the more common culprit. Inspect the retrieved chunks first.
- **Setting `k` too high.** Padding the prompt with marginal chunks dilutes the signal, raises cost, and worsens "lost in the middle." Precision over volume.
- **Never using a score threshold.** Without one, the retriever always returns `k` chunks even for absent topics, causing confident hallucinations. A threshold lets the system fail honestly.
- **Guessing the threshold value.** It depends on your data, model, and whether the store reports similarity or distance. Set it from measured score distributions.
- **Using pure semantic search for code-heavy or identifier-heavy data.** Dense retrieval under-ranks exact tokens. Use hybrid search.
- **Ignoring redundancy.** Plain similarity can return near-duplicate chunks, wasting context on broad questions. Use MMR.
- **Letting the BM25 and dense indexes drift out of sync.** Rebuild both together on every re-ingestion.
- **Treating retrieval choices as permanent.** `k`, threshold, `lambda_mult`, and ensemble weights are all tunable — and should be tuned by measurement, not intuition.

5.17 Best Practices

- **Inspect retrieved chunks when debugging** — the answer's quality is bounded by them.
- **Keep `k` in the 3-8 range** and tune it with metrics.
- **Always apply a score threshold** so the system can return nothing and say "I don't know."

- **Set the threshold empirically** from should-match and should-not-match score distributions.
- **Use MMR for broad, multi-faceted, or repetitive content** to ensure result diversity.
- **Default to hybrid search** (dense + BM25) — it covers both meaning and exact terms with little downside.
- **Weight the ensemble to your domain** — more BM25 for identifier-heavy data, more dense for conceptual prose.
- **Keep sparse and dense indexes synchronised**, and combine retrieval with metadata filtering.

5.18 Real-World Applications

- **Customer-support assistants** — hybrid search matches both the user's paraphrased problem and exact error codes or product names.
- **Documentation and developer Q&A** — BM25 catches exact API names, flags, and identifiers that semantic search alone under-ranks.
- **Enterprise knowledge assistants** — score thresholds let the bot honestly say "that is not in our documents" instead of inventing policy.
- **Legal and compliance search** — hybrid retrieval matches both legal concepts and exact statute or clause numbers.
- **E-commerce and product search** — MMR diversifies results so a query returns a *range* of relevant products, not five variants of one.
- **Research assistants** — MMR surfaces a breadth of perspectives on a topic rather than several papers making the identical point.

5.19 Mini Project: A Retriever Comparison Lab

Make the trade-offs of this module concrete by building a small comparison lab:

1. Using Module 2's `ingest.py`, build a knowledge base from a document set that contains both conceptual prose and some exact identifiers (codes, names, version numbers).
2. Construct all four retrievers from this module over that knowledge base: plain similarity, score-threshold, MMR, and hybrid ensemble.
3. Prepare three categories of test query: (a) a **broad** question with multiple facets, (b) a question containing an **exact identifier**, and (c) a question about something **absent** from the knowledge base.
4. Run every query through every retriever and lay the results side by side in a table.

5. **Analyse.** Confirm that MMR returns more diverse chunks than plain similarity on the broad query; that hybrid beats pure semantic on the exact-identifier query; and that the threshold retriever returns nothing for the absent topic.
6. **Tune.** For the hybrid retriever, try weights `[0.2, 0.8]`, `[0.5, 0.5]`, and `[0.8, 0.2]` on the exact-identifier query and observe how the ranking shifts. For the threshold retriever, find a `score_threshold` that accepts your good queries while rejecting the absent-topic one.
7. **Write up** a short paragraph recommending which retriever (and which settings) you would deploy for *this* knowledge base, with reasons.

This lab turns the abstract trade-offs into something you have measured with your own queries — exactly the empirical mindset Module 9 will formalise.

5.20 Chapter Summary

- A **retriever** takes a text query and returns relevant chunks. It is the query phase's search step, and answer quality is bounded by it — the generator can only reason over what the retriever provides.
- **Plain similarity search** returns the k nearest chunks by embedding distance. `k` is a precision-versus-recall trade-off; 3–8 is the usual range. More chunks is not safer — it dilutes signal and raises cost.
- Plain similarity **always returns `k` chunks**, even when nothing is relevant. A **similarity score threshold** discards weak matches so the retriever can return nothing, letting the system honestly say "I don't know." Set the threshold from measured score distributions.
- **Maximal Marginal Relevance (MMR)** selects chunks that are relevant to the query *and* diverse from each other, solving the redundancy problem. `lambda_mult` dials between pure relevance (1.0) and pure diversity (0.0); ~0.5 is balanced. Use MMR for broad or repetitive content.
- **Dense retrieval** (semantic, embedding-based) matches *meaning* — synonyms, paraphrase, intent — but under-ranks exact tokens. **Sparse retrieval** (keyword, e.g. **BM25**) matches *exact words* — codes, IDs, names — but misses paraphrased concepts. They fail in opposite situations.
- **BM25** scores chunks by term frequency, inverse document frequency (rarer words matter more), and document-length normalisation.
- **Hybrid search** runs dense and sparse retrieval together and merges them. Because their raw scores are not comparable, results are fused by **Reciprocal Rank Fusion (RRF)** — by rank position, not score.

- An **ensemble retriever** combines several retrievers with weights; a `BM25Retriever` + vector retriever ensemble is the standard way to build hybrid search in LangChain. Hybrid search is a high-return, low-risk production default.
- Key optimisation levers: tune `k`, use thresholds, use MMR where redundancy is likely, default to hybrid, filter by metadata, fix chunking upstream, and measure everything.

5.21 Practice Exercises

1. **Conceptual.** Explain, in under 120 words, why the basic top- k similarity retriever can cause a RAG system to hallucinate, and how a score threshold fixes it.
2. **Trade-off.** Describe one concrete scenario where setting `k=2` gives better answers than `k=10`, and one where the reverse is true. Explain both.
3. **MMR.** A user asks "tell me about the company's leave policies." A plain retriever returns four chunks all about annual leave. Explain step by step how MMR with `lambda_mult=0.5` would produce a better result set.
4. **Dense vs sparse.** For each query, state whether dense or sparse retrieval is more likely to find the right chunk, and why: (a) "how do I cancel my subscription"; (b) "what does error code TX-9931 mean"; (c) "ways to get a refund"; (d) "the policy for employee Arjun Mehta".
5. **Code.** Run `module5_retrieval.py`. Confirm that MMR returns more diverse chunks than plain similarity for "annual leave," and that the threshold retriever returns nothing for the stock-price query.
6. **Code — hybrid.** Add a chunk that is *conceptually* about login problems but never uses the string "E-77." Query "fix error E-77" with the dense retriever and then the hybrid retriever, and explain the difference in results.
7. **Threshold tuning.** Using `similarity_search_with_score`, gather scores for five queries you know have answers and five you know do not. Propose a `score_threshold` that separates them, and justify it.

5.22 Interview Questions

BEGINNER

Q1. What is a retriever in a RAG system?

A component that takes a text query and returns the document chunks most worth showing the language model. It performs the "search" step of the query phase, and answer quality is bounded by it.

Q2. What does the parameter `k` control, and why not just set it very high?

`k` is the number of chunks the retriever returns. Setting it very high pads the prompt with marginally relevant chunks, which dilutes the real signal, raises token cost, and worsens the "lost in the middle" effect. Retrieval is about precision, not volume.

Q3. What is a similarity score threshold and what problem does it solve?

It is a minimum similarity below which a retrieved chunk is discarded. It solves the problem that plain similarity search always returns k chunks even when nothing is relevant; with a threshold, the retriever can return nothing, so the system can honestly say "I don't know" instead of hallucinating.

Q4. What is the difference between dense and sparse retrieval?

Dense retrieval matches by *meaning* using embeddings (handles synonyms and paraphrase). Sparse retrieval, such as BM25, matches by *exact words* (handles codes, IDs, and names). They fail in opposite situations.

INTERMEDIATE**Q5. What is Maximal Marginal Relevance and when would you use it?**

MMR selects chunks by balancing two criteria: relevance to the query and difference from chunks already chosen. It solves the redundancy problem, where plain similarity returns several near-duplicate chunks. A parameter (`lambda_mult`) dials between pure relevance and pure diversity. Use MMR for broad, multi-faceted questions or knowledge bases with repetitive content.

Q6. What is BM25 and what are its strengths and weaknesses?

BM25 is a classic keyword-ranking algorithm scoring chunks by term frequency, inverse document frequency (rarer words count more), and document-length normalisation. Its strength is exact matching of distinctive tokens — codes, identifiers, names, rare jargon. Its weakness is that it has no notion of meaning, so it misses synonyms and paraphrased concepts.

Q7. What is hybrid search and why does it outperform pure semantic search?

Hybrid search runs both dense (semantic) and sparse (BM25) retrieval and merges the results. It outperforms pure semantic search because the two methods fail in opposite situations — dense misses exact tokens, sparse misses paraphrased meaning — so combining them is robust to both kinds of query.

Q8. The two retrievers in a hybrid setup produce different score scales. How are their results combined?

With Reciprocal Rank Fusion (RRF), which combines results by their *rank position* in each list rather than by raw score. This works because rank is comparable across methods even when raw scores (e.g. a 0–1 cosine similarity versus an unbounded BM25 score) are not.

ADVANCED**Q9. How would you choose and tune a score threshold for a production retriever?**

Empirically. First determine whether the vector store reports a similarity (higher is better) or a distance (lower is better). Then collect scores from a batch of queries known to have good answers and a batch known to have none, examine both score distributions, and pick a cut-off in the gap between them. Too high rejects genuine answers (low recall); too low lets irrelevant chunks through (low precision). Re-validate it whenever the embedding model or corpus changes.

Q10. When would you weight a hybrid ensemble toward BM25, and when toward the dense retriever?

Weight toward BM25 when the domain is rich in exact identifiers — error codes, part numbers, API names, proper names — where precise token matching matters most. Weight toward the dense retriever when the content is conceptual prose and users paraphrase freely, so meaning matters more than exact words. The right weights are found by measuring retrieval quality on representative queries, not by intuition.

Q11. A RAG system gives a confident but wrong answer to a question whose information is not in the knowledge base. Trace the retrieval-side causes and fixes.

The retriever almost certainly returned its k least-distant chunks even though none was actually relevant, because plain similarity search always returns k results. The model then answered from irrelevant context. Fixes: add a similarity score threshold so the retriever returns nothing when all candidates are weak; and ensure the prompt instructs the model to answer only from context and to say it does not know otherwise (Module 1). Together these let the system fail honestly instead of confidently.

SCENARIO-BASED

Q12. You build a RAG assistant over a software product's documentation. It answers conceptual "how do I..." questions well, but when users paste an exact error code it often returns generic troubleshooting pages that never mention that code. What is happening and how do you fix it?

The system is using pure dense (semantic) retrieval. Embeddings capture the overall *meaning* of the query but treat a rare, exact token like an error code as just one minor contributor, so a chunk that is "generally about errors" can outrank the chunk containing the precise code. The fix is hybrid search: add a BM25 sparse retriever and combine it with the dense retriever in an ensemble. BM25 matches the exact code reliably — and because the code is a rare token, inverse document frequency makes it score strongly. For a heavily code-laden domain, consider weighting the ensemble somewhat toward BM25, and validate the weights on real queries.

Q13. A user reports that asking your bot a broad question — "summarise all our remote-work policies" — yields an answer that only covers one narrow aspect, even though the knowledge base documents several. What is the likely retrieval cause and remedy?

The likely cause is redundancy in plain similarity retrieval: the densest, most repeated section of the knowledge base (say, the part about equipment stipends) dominates the top-*k* results, so the retriever returns several chunks all about that one aspect and the model never sees the others. The remedy is MMR retrieval, which balances relevance with diversity and so returns one strong chunk per facet, giving the model the breadth needed to summarise the whole policy. Increasing *k* modestly and ensuring chunking aligns with policy sections (Module 2) can also help.

END OF MODULE 5

Your retriever is now solid: it searches by meaning and by exact words, it can refuse to answer when nothing fits, and it avoids redundant results. But it still treats every query exactly as the user typed it, and it returns whole chunks as-is. **Module 6 — Advanced Retrieval Techniques** goes further: contextual compression to trim retrieved chunks down to only what matters, the parent document retriever, the self-query retriever that turns natural language into metadata filters automatically, and multi-query retrieval and query rewriting that improve the question *before* it is ever searched.